

Test Driven Development (TDD)

Relembrando...

Testes de software mostram a presença de bugs,
mas não a sua ausência.

E. W. Dijkstra

Testes em Métodos Ágeis

- ↴ Preferência por Testes Automatizados

- ↴ Testes Unitários são priorizados:

 - ↴ Menor custo

 - ↴ Pela técnica

 - ↴ Pelo momento de execução

 - ↴ Mais rápidos

- ↴ Tendem a ser implementados antes do código

 - ↴ *Test Driven Development* (TDD)

TDD

↴ Prática de programação proposta pelo XP

↴ Benefícios:

- ↵ Evita que os desenvolvedores esqueçam dos testes
- ↵ Favorece a escrita de código com testabilidade
- ↵ Cobertura dos testes tende a ser maior
- ↵ Favorece ao *Clean Code* - o desenvolvedor é o primeiro cliente do seu próprio código

TDD

- ↴ Podemos ter 3 estados neste processo:
 - ↴ Teste com **falha**
 - ↴ Teste **OK**
 - ↴ **Refatoração**

Exemplo de um carrinho de compras: Vermelho

```
@Test
void testAddGetTotal() {
    Book b1 = new Book("book1", 10, "1");
    Book b2 = new Book("book2", 20, "2");
    ShoppingCart cart = new ShoppingCart();
    cart.add(b1);
    cart.add(b2);
    assertEquals(30.0, cart.getTotal());
}
```

```
public class Book {  
    public String title;  
    public double price;  
    public String isbn;  
  
    public Book(String title, double price, String isbn) {  
        this.title = title;  
        this.price = price;  
        this.isbn = isbn;  
    }  
}  
  
public class ShoppingCart {  
  
    public ShoppingCart() {}  
  
    public void add(Book b) {}  
  
    double getTotal() {  
        return 0.0;  
    }  
}
```

Implementação provisória

Verde

```
public class ShoppingCart {  
    public ShoppingCart() {}  
    public void add(Book b) {}  
    double getTotal() {  
        return 30.0;  
    }  
}
```

Como um primeiro teste OK, mas que precisa de uma implementação funcional.

Implementando...

```
public class ShoppingCart {  
    private ArrayList<Book> items;  
  
    private double total;  
  
    public ShoppingCart() {  
        items = new ArrayList<Book>();  
        total = 0.0;  
    }  
  
    public void add(Book b) {  
        items.add(b);  
        total += b.price();  
    }  
  
    double getTotal() {  
        return total;  
    }  
}
```

Amarelo: podemos refatorar e melhorar o código?

```
public class Book {  
    public String title;  
    public double price;  
    public String isbn;  
  
    public Book(String title, double price, String isbn) {  
        this.title = title;  
        this.price = price;  
        this.isbn = isbn;  
    }  
}
```

Por exemplo, encapsular esses campos

Referências

- ↓ Gerard Meszaros. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley, 2007.
- ↓ Kent Beck, Erich Gamma. Test-infected: programmers love writing tests. Java Report, 3(7):37-50, 1998.
- ↓ Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020.
- ↓ Wazlawick, R. S. Engenharia de Software. Elsevier, 2013.